

Name: AAVISHKAR SINGH AAVISHKAR SINGH | Reg. No: RA2511003011024

Unit 3 : Session 8 : SLO 2 — Restoring vs Non-Restoring Division

■ Section C: Simulate the Algorithm (Apply): 0100 (4) ÷ 0010 (2) — non-restoring

- Init: $A = 0000$, $Q = 0100$, $M = 0010$
- Step 1: shift → $A = 0000$, $Q = 1000$; $A = A - M = 1110 (< 0)$, $Q_0 = 0 \rightarrow A = 1110$, $Q = 1000$
- Step 2: shift → $A = 1101$, $Q = 0000$; $A < 0 \rightarrow A = A + M = 1111 (< 0)$, $Q_0 = 0 \rightarrow A = 1111$, $Q = 0000$
- Step 3: shift → $A = 1110$, $Q = 0000$; $A < 0 \rightarrow A = A + M = 0000 (\geq 0)$, $Q_0 = 1 \rightarrow A = 0000$, $Q = 0001$
- Step 4: shift → $A = 0000$, $Q = 0010$; $A \geq 0 \rightarrow A = A - M = 1110 (< 0)$, $Q_0 = 0 \rightarrow A = 1110$, $Q = 0010$
- Final correction: $A = A + M = 0000$. Quotient = 0010 (2), Remainder = 0000 (0) ✓

■ Section D: Debrief — comparing logic flow and efficiency

- Efficiency: non-restoring removes the restore step, so every cycle costs exactly one addition/subtraction plus a shift — fewer operations and simpler control than restoring division.
- Correction at the end: a negative final A is not a valid remainder, so the divisor is added back once to fix it.
- Quotient-bit assignment: restoring sets the bit only after a possible restore; non-restoring sets it straight from the sign of A (1 if $A \geq 0$, 0 if $A < 0$).
- Logic flow: restoring = shift → subtract → if negative restore (add back) → set bit; non-restoring = shift → add or subtract depending on the sign of A → set bit → single correction at the end.